TASK 1: EMPLOYEE ID GENERATOR

Aim

To develop a Python program using a class, constructor, class variables, instance variables, an instance method, a list, and loops to accept details of multiple employees, generate employee IDs, and display the records together.

OOP Concepts Used

Concept	Implementation
Class	Employee
Class (static) variables	company_name, employee_count
Instance variables	empoyee_name, dept
Constructor	__init__(self, name, department)
Instance method	generate_employee_id(self)
Local variable	employee_id
Objects and list	emp objects stored in employ

Program

```
class Employee:
    comp_name = "Kaizentrix Global Solution"
    emp_count = 1000

    def __init__(self, name, department):
        self.emp_name = name
        self.dept = department

    def emp_generateid(self):
        Employee.emp_count += 1
        employee_id = "EMP" + str(Employee.emp_count)
        return employee_id

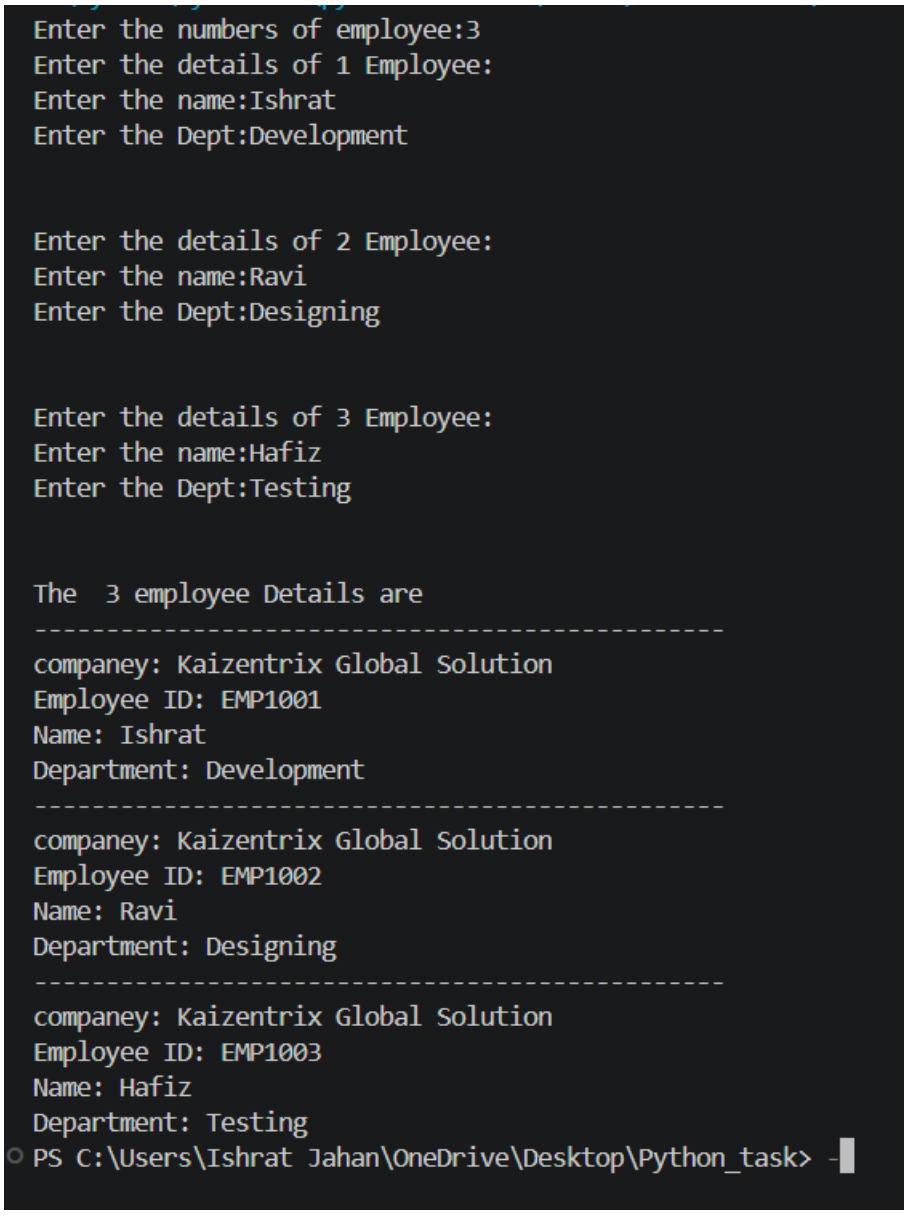
n = int(input("Enter the number of employees: "))
if n>0:
    employ=[ ]
    for i in range(1,n+1):
       print("Enter the details of",i,"Employee:")
       name=input("Enter the name:")
       dept=input("Enter the Dept:")
       print("\n")
       emp=Employee(name,dept)
       employ.append(emp)

    print("The ",n,"employee Details are ")
    for emp in employ:
        print("-------------------------------------------------")
        print("companey:",emp.comp_name)
        print("Name:",emp.emp_name)
```

```
        print("Department:",emp.dept)
        print("Emp_id:",emp.emp_generateid())
        print("-----------------------------------------------")

else:
    print("Enter positive count")
```

Sample Input and Output



Working

The program first asks for the number of employees. The first for loop accepts each employee's name and department, creates an Employee object, and stores it in the employ list. After input is complete, the second loop displays every employee's details. The emp_generateid() method increments the shared employee count and returns an ID such as EMP1001.

Result

The Employee ID Generator successfully accepts details for multiple employees and displays their records with sequential employee IDs.

Note: For this assignment, the ID-generation method is called once per employee. Calling it again for the same object generates a new ID.

I used a list to store multiple employee objects in a single collection. This allows the program to collect all employee details first and display them together at the end.

The list also helps to:

- Store any number of employee objects dynamically.
- Avoid creating separate variables like emp1, emp2, and emp3.
- Access and display employee details using a for loop.
- Reduce code repetition and improve program efficiency.